PROJECT REFERENCE · JSAN GEOSALES 360

The road from login to a working field-sales system

A start-to-finish walk through what you've built, a checklist to confirm it actually works end to end, and the steps only you can take from here.

All 10 phases complete

3-role permission system live

Backend permission enforcement partial — see roadmap

The lifecycle, step by step

01

Every module exists to move one piece of business through this pipeline — from a rep spotting a shop on the street to it showing up as real revenue on the Dashboard. This is the order the system actually expects things to happen in.

1

Login

Sign in with a role

JWT login gates the whole app. The role attached to the account — Administrator, Sales Manager, Sales Executive, Field Sales Representative, or GIS Analyst — decides which sidebar items and dashboard widgets even appear, per `rolePermissions.js`.

2

Discovery

Log a candidate found in the field

A rep records a business they've spotted — name, address, phone, source. Duplicate detection runs automatically against existing Accounts, Leads, and other candidates, combining name/phone/address matching with real spatial distance.

3

Discovery → Leads

Review, approve, convert

A manager approves or rejects the candidate. An approved, non-duplicate candidate converts into a real Salesforce Lead with one click — no re-typing.

4

Territories

Draw boundaries, assign coverage

Draw each territory's real boundary on the map. "Recalculate Territory Assignments" then runs actual point-in-polygon matching, assigning every Account, Lead, and Discovery Candidate to whichever boundary contains its coordinates.

5

GIS Map

Plan the day from the map

The central hub — search every Account, Lead, and Opportunity, filter by territory, priority, or type, and read the AI Executive Dashboard's real, live-computed analytics (not placeholder numbers, as of the latest fix).

6

Route Planning

Generate a real route

"Generate Route" sends the territory's pending stops to OpenRouteService, which returns an optimized visit order following actual roads — real distance and ETA, not a straight line between pins.

7

Field Visits

Check in, log the outcome, check out

On site, the rep checks in — the browser's real GPS position is measured against the record's saved coordinate, and check-in only succeeds within a 150m geofence. They record an outcome and notes, then check out, writing a real `Field_Visit__c` record and updating the Account or Lead's validation status.

8

Opportunities

Turn a validated account into a deal

Once an Account is validated, an Opportunity can be opened against it and carried through the standard sales stages to Closed Won or Closed Lost.

9

Dashboard

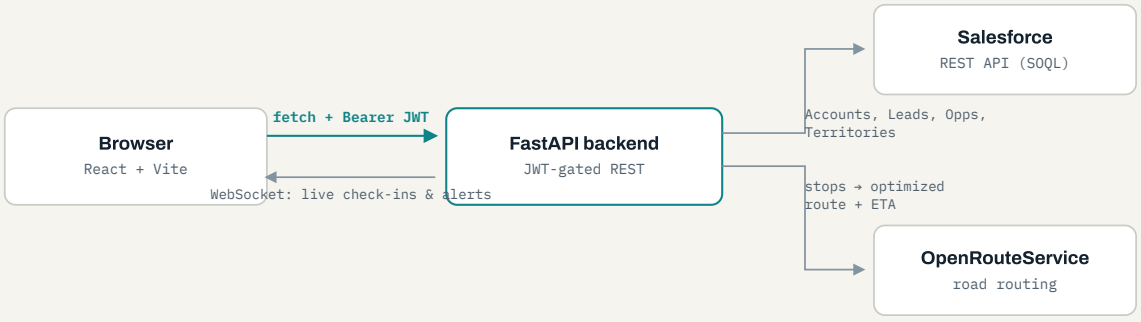
See the whole business at once

The Dashboard module rolls up every one of the above — Accounts, Leads, Opportunities, Discovery, Territories, Routes, Field Visits — into one company-wide view: pipeline value, win rate, conversion funnel, territory coverage, route status.

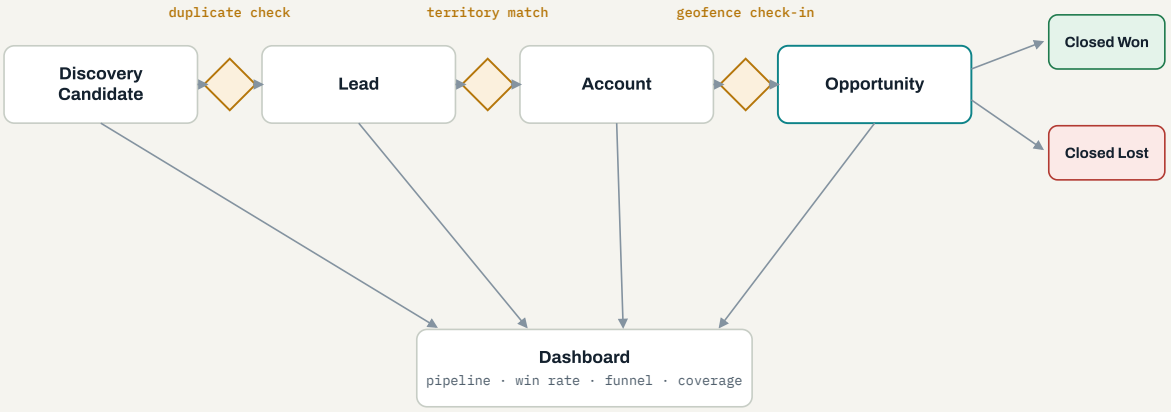
How the pieces actually talk

02

Two mechanisms underneath the lifecycle above: how a browser request actually reaches Salesforce and comes back live, and where the automatic checks — duplicate detection, territory matching, geofencing — sit inside the record pipeline.



Every request carries a **JWT** issued at login; the backend is the only thing that ever talks to Salesforce or OpenRouteService directly, and pushes live activity back to every open tab over a **WebSocket**, not polling.



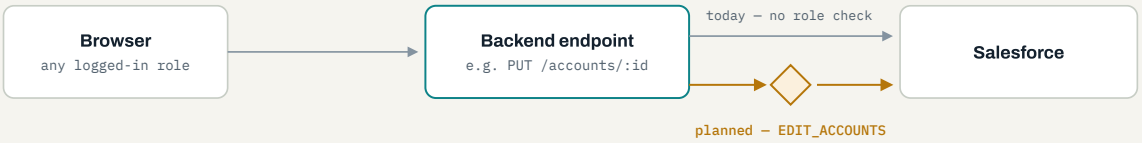
The three amber gates are the only places a record is automatically stopped or redirected — everything else is a straight conversion. **Territory match** is a real point-in-polygon test against a drawn boundary (falling back to a compass-direction sector when

a territory has no boundary yet); **geofence check-in** is a live GPS distance check against the record's saved coordinate.

Where this goes next

03

Everything above is how the app works *today*, verified live. This is the gap between that and where it's headed — five concrete, known differences between what the UI already enforces and what the system will enforce end to end.



Right now, exactly **2 of ~90** backend endpoints check the caller's role — the two User Roles admin endpoints. Every other endpoint (Accounts, Leads, Territories, Field Visits, ...) only checks that the request carries *a* valid login, same as it did before the 3-role rework — the new permission model lives entirely in the frontend so far. Closing that gap means adding the same kind of permission-key check this diagram shows to every write endpoint the frontend already restricts.

CAPABILITY	TODAY	PLANNED
Backend permission checks	2 of ~90 endpoints (User Roles only)	**Every endpoint checked against its matching permission key**
"My assigned records" scoping	Unlocks the module only — a Field User sees the same full Accounts/Leads list as everyone else	**Filtered to records actually owned by that field user, once a login is linked to a Salesforce Owner ID**
Live push events	`field_visit_updated` and `account_updated` fire for real; `gis_updated` and `alert` are wired on the frontend but never sent	**Either wired up for real triggers, or the two dead handlers removed**

CAPABILITY	TODAY	PLANNED
Generated routes	Live only in the browser tab's session state	**Persisted per rep and date, so "my route today" survives a fresh login**
Territory boundaries	Some territories still fall back to the compass-direction sector	**Every territory has a real hand-drawn boundary**

Confirm the goal is actually working

04

A working demo and a working tool look identical from the login screen. These are the checks that tell them apart — each one exercises a real write to Salesforce, not just a page that loads.

ACCESS

- [ ] **Log in as two different roles** (e.g. `SALES_MANAGER` and `FIELD_USER`).
  Confirm the sidebar genuinely differs between them, matching `rolePermissions.js` — not just that login succeeds.

CORE RECORDS

- [ ] **Create one test Account or Lead** without a real address.
  Confirm it gets an auto-assigned Hyderabad-region coordinate and appears as a pin on the GIS Map within a refresh.

- [ ] **Log a Discovery Candidate** with a name or phone close to a record that already exists.
  Confirm the duplicate check actually flags it before you approve it.

TERRITORIES

- [ ] **Open GIS Map → Territory Boundaries.**
  Confirm all four territories (HYD-NORTH / SOUTH / WEST / EAST) have a real drawn boundary, not just a small test shape.

- [ ] **Run "Recalculate Territory Assignments."**
  The accounts/leads/candidates-reassigned counts it returns should roughly match how many records actually fall inside a drawn boundary.

THE FIELD LOOP

- [ ] **Check in on a record within ~150m of its saved coordinate** (a real device, or Chrome DevTools' location override).
  Confirm the distance shown is accurate, log an outcome, check out, then verify a new `Field_Visit__c` exists in Salesforce with real check-in/check-out times.

- [ ] **Generate a route** for a territory with two or more pending stops.
  The drawn line should follow real streets, not cut a straight diagonal across the map.

THE NUMBERS

- [ ] **Spot-check one Dashboard KPI against Salesforce directly** — e.g. count "Closed Won" Opportunities yourself.
  It should match the card exactly.

- [ ] **Open two browser tabs, check out a visit in one.**
  Confirm the other tab's Live AI Alerts / Activity Feed updates on its own — that's the real-time WebSocket layer, not a page refresh.

What still needs you

05

Everything below is outside what I can do from inside the codebase — credentials, real-world data entry, and decisions about scope.

Real staff logins

BEFORE GO-LIVE

- `APP_USERS` on the backend currently holds one real Administrator and whatever accounts were set up for testing.
- Generate real ones with `backend/scripts/make_user.py`

Finish the territory boundaries

BEFORE GO-LIVE

- HYD-SOUTH, HYD-WEST, and HYD-EAST still need their real boundaries drawn — only HYD-NORTH has more than a small test shape.

`<username> <password> <role>` — role must be exactly `ADMIN`, `SALES_MANAGER`, or `FIELD_USER`.

- After adding or changing a role in Render's `APP_USERS`, that person must sign out and log back in — their existing token isn't re-checked against the new value.

- Re-run "Recalculate Territory Assignments" once they're all drawn.

Confirm deployment config

WORTH A PASS

- Render (backend) and Vercel (frontend) env vars should match what's actually live — see the reference table below.
- `JWT_SECRET_KEY` should be a long random value; rotate the Salesforce connected-app secret if it's ever been shared or pasted anywhere.

Know the free-tier limits

WORTH KNOWING

- Render's free backend tier cold-starts after inactivity — the first request after idle time can be slow.
- OpenRouteService's free key caps at 2,500 route requests/day.

A pass on leftover test data

WORTH A PASS

- A handful of sample Leads and one sample Opportunity owner are hard-excluded by ID in the backend, left over from earlier org setup.
- Worth a look now that real usage has started, in case anything else needs cleaning up.

Read the full handover docs

YOUR CALL

- This page is a field guide, not the full manual — `docs/` now has a formal README, architecture doc, Salesforce field reference, deployment runbook, 49 test cases, and a Postman collection covering all ~90 endpoints.
- See §03 above for the known gaps between what's built and what's planned.

Environment quick reference

06

What the backend expects in `.env` (or the Render dashboard). Full context for each lives in `backend/.env.example`.

VARIABLE	WHAT IT'S FOR
`SF_LOGIN_URL / SF_CLIENT_ID / SF_CLIENT_SECRET`	The Salesforce connected app's client-credentials login.
`FRONTEND_URL`	The deployed frontend's origin, for CORS.
`JWT_SECRET_KEY`	Signs login tokens. Long, random, never reused elsewhere.
`ACCESS_TOKEN_EXPIRE_MINUTES`	Login session length. Defaults to 480 (8 hours).
`APP_USERS`	The real login accounts — JSON array of username / password hash / role.
`ORS_API_KEY`	OpenRouteService key powering "Generate Route."
`VITE_API_BASE_URL (frontend)`	The backend's URL — everything else (including the WebSocket) derives from this.